

Interpolacja funkcji

Wzór interpolacyjny Newtona

Ewa Kasprzak

8 grudnia 2024

1 Wstęp

Interpolacja funkcji to metoda numeryczna, która umożliwia skonstruowanie funkcji przybliżającej (zwykle wielomianu), przechodzącej dokładnie przez zestaw danych wejściowych, zwanych węzłami interpolacji. Celem interpolacji jest oszacowanie wartości funkcji w punktach leżących pomiędzy znanymi danymi. W przeciwieństwie do aproksymacji, interpolacja wymaga, aby funkcja interpolująca dokładnie przechodziła przez wszystkie punkty danych.

Interpolacja wielomianowa

Interpolacja wielomianowa polega na znalezieniu wielomianu $P_n(x)$ stopnia n , który przechodzi przez $n + 1$ węzłów interpolacji $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$. Warunek interpolacji wyraża się równaniem:

$$P_n(x_i) = y_i \quad \text{dla} \quad i = 0, 1, \dots, n.$$

Wielomian $P_n(x)$ można zapisać w postaci ogólnej:

$$P_n(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n,$$

gdzie współczynniki $a_0, a_1, \dots, a_n$ wyznacza się tak, aby spełniały warunki przechodzenia przez węzły.

Jeśli liczby $x_0, x_1, \dots, x_n$ są parami różne, to istnieje dokładnie jeden wielomian $P_n(x)$, który spełnia powyższe warunki.

Konstrukcja wielomianu interpolacyjnego

Wielomian interpolacyjny można wyznaczyć w sposób iteracyjny, wykorzystując tzw. **wzór Newtona**, który ma postać:

$$P_k(x) = c_0 + c_1(x - x_0) + c_2(x - x_0)(x - x_1) + \dots + c_k(x - x_0)(x - x_1) \cdots (x - x_{k-1}),$$

lub w bardziej zwartej formie:

$$P_k(x) = \sum_{i=0}^k c_i \prod_{j=0}^{i-1} (x - x_j),$$

gdzie współczynniki c_i są wyznaczane na podstawie ilorazów różnicowych.

Metoda konstrukcji wielomianu przy użyciu wzoru Newtona jest teoretycznie prosta, ale stosunkowo kosztowna obliczeniowo. Z tego powodu w praktyce często korzysta się z **tablic ilorazów różnicowych**, które pozwalają na bardziej efektywne obliczanie współczynników c_i . Ilorazy różnicowe upraszczają obliczenia i redukują złożoność procesu konstrukcji wielomianu.

2 Iloraz różnicowy

Ilorazem różnicowym rzędu k dla funkcji f nazywamy wartość:

$$c_k = f[x_0, x_1, \dots, x_k].$$

Wzór interpolacyjny Newtona można zatem przedstawić jako:

$$p(x) = \sum_{k=0}^n f[x_0, \dots, x_k] \prod_{j=0}^{k-1} (x - x_j).$$

Dla ilorazów różnicowych pierwszego i drugiego rzędu jawne wyrażenia ilorazu różnicowego są następujące:

$$f[x_0] = f(x_0), \quad f[x_0, x_1] = \frac{f(x_1) - f(x_0)}{x_1 - x_0}.$$

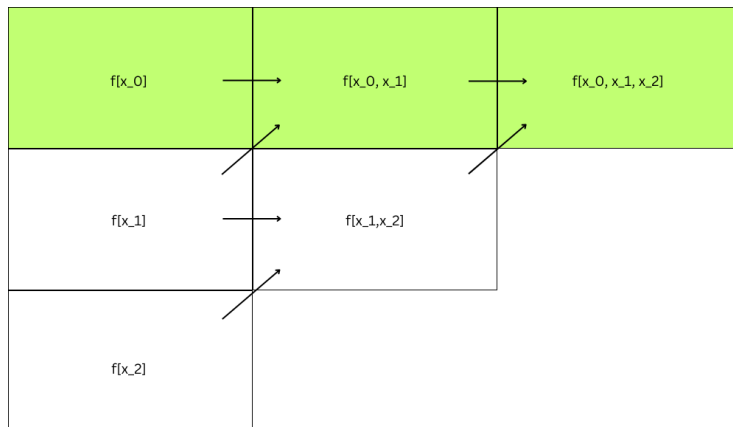
Dla ilorazów wyższego rzędu definiuje się rekurencyjną zależność:

$$f[x_0, \dots, x_k] = \frac{f[x_1, \dots, x_k] - f[x_0, \dots, x_{k-1}]}{x_k - x_0}.$$

Pozwala to na stworzenie tzw. **tablicy ilorazów różnicowych**:

$$\begin{array}{c|c|c|c|c} x_0 & f[x_0] & f[x_0, x_1] & f[x_0, x_1, x_2] & \cdots \\ x_1 & f[x_1] & f[x_1, x_2] & \cdots & \\ x_2 & f[x_2] & \cdots & & \\ \vdots & \vdots & & & \\ x_n & f[x_n] & & & \end{array}$$

Wizualny sposób tworzenia tablicy został pokazany na rysunku poniżej:



Rysunek 1: Konstrukcja tablicy ilorazów różnicowych

Interesujące nas współczynniki znajdują się w pierwszym wierszu tabeli. Formalnie konstrukcję tabeli można przedstawić jako:

$$c_{ij} = f[x_i, \dots, x_{i+j}].$$

Wtedy tablica ilorazów różnicowych wygląda następująco:

x_0	c_{00}	c_{01}	c_{02}	$\dots$
x_1	c_{10}	c_{11}	$\dots$	
x_2	c_{20}	$\dots$		
$\vdots$	$\vdots$			
x_n	c_{n0}			

A wzór rekurencyjny można wyrazić jako:

$$c_{ij} = \frac{c_{i+1,j-1} - c_{i,j-1}}{x_{i+j} - x_i}.$$

Algorytm tworzący tablicę ilorazów różnicowych

W klasycznym podejściu, opartym na przedstawionym wyżej wzorze rekurencyjnym, do obliczania ilorazów różnicowych tworzona jest tablica o rozmiarze $n \times n$, aby pomieścić wszystkie wartości ilorazów różnicowych dla wszystkich par punktów. Jednak w przedstawionym algorytmie, zamiast tworzyć pełną tablicę, modyfikujemy tylko jedną tablicę jednowymiarową $\mathbf{fx}$.

Zamiast tworzyć tablicę dwuwymiarową, algorytm iteracyjnie oblicza wartości ilorazów różnicowych i aktualizuje tylko odpowiednie elementy w jednowymiarowej tablicy $\mathbf{fx}$, co skutkuje znaczną oszczędnością pamięci (z $O(n^2)$ na $O(n)$).

```

function ilorazyRoznicowe(x::Vector{Float64}, f::Vector{Float64})
    n = length(x) - 1
    fx = copy(f)
    for j in 2:n+1
        for i in n+1:-1:j
            fx[i] = (fx[i] - fx[i-1]) / (x[i] - x[i-j+1])
        end
    end
    return fx
end

```

Złożoność czasową algorytmu możemy zapisać jako:

$$T(n) = \sum_{j=2}^{n+1} (n+1-j) = \sum_{j=1}^n j = \frac{n(n+1)}{2}$$

Zatem algorytm wykonuje $O(n^2)$ operacji, a jego złożoność czasowa jest proporcjonalna do kwadratu liczby punktów n .

3 Efektywne obliczanie wartości wielomianu Newtona w punkcie t

Klasyczny algorytm Hornera jest wykorzystywany do efektywnego obliczania wartości wielomianu o postaci:

$$p(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n.$$

Można go zapisać w formie zagnieżdżonej jako:

$$p(x) = a_0 + x(a_1 + x(a_2 + \dots + x(a_{n-1} + a_nx) \dots)).$$

To podejście pozwala na obliczanie wartości wielomianu w sposób iteracyjny, redukując liczbę mnożeń.

Uogólniony algorytm Hornera dla wielomianu Newtona

W przypadku wielomianu interpolacyjnego Newtona, jego postać wygląda następująco:

$$p(x) = f[x_0] + (x-x_0)f[x_0, x_1] + (x-x_0)(x-x_1)f[x_0, x_1, x_2] + \dots + (x-x_0) \dots (x-x_{n-1})f[x_0, \dots, x_n].$$

Uogólniony algorytm Hornera rozszerza ten klasyczny pomysł, przekształcając powyższy wzór do postaci:

$$p(x) = f[x_0] + (x-x_0)(f[x_0, x_1] + (x-x_1)(f[x_0, x_1, x_2] + \dots + (x-x_{n-1})f[x_0, \dots, x_n])).$$

Wyjaśnienie działania algorytmu

1. **Postać wielomianu Newtona:** Wielomian interpolacyjny Newtona jest wyrażony w postaci sumy składników, w których każdy składnik zawiera ilorazy różnicowe $f[x_0, \dots, x_k]$ oraz różnice $(x - x_0), (x - x_1), \dots, (x - x_{k-1})$.
2. **Przekształcenie do formy Hornera:** Zamiast obliczać każdy składnik osobno, możemy zastosować uogólniony algorytm Hornera, który iteracyjnie przekształca te składniki do formy zagnieżdżonej. Rozpoczynamy od ostatniego ilorazu różnicowego $f[x_0, \dots, x_n]$, a następnie iteracyjnie obliczamy wcześniejsze ilorazy różnicowe, uwzględniając kolejno różnice $(x - x_k)$.
3. **Iteracyjne obliczanie wartości:** Zaczynamy od ostatniego ilorazu różnicowego $f[x_0, \dots, x_n]$, a następnie w pętli (od $n - 1$ do 0) obliczamy kolejne wartości $w_k(x)$ za pomocą wzoru rekurencyjnego:

$$w_k(x) = f[x_0, \dots, x_k] + (x - x_k)w_{k+1}(x).$$

4. **Ostateczna wartość wielomianu:** Ostateczną wartość wielomianu Newtona w punkcie $x = t$ otrzymujemy, obliczając $w_0(t)$, co daje wartość $N_n(t)$.

Implementacja algorytmu

Poniżej przedstawiona jest implementacja algorytmu obliczającego wartość wielomianu Newtona w punkcie t za pomocą uogólnionego algorytmu Hornera:

```
function warNewton(x::Vector{Float64}, fx::Vector{Float64}, t::Float64)
    n = length(x)
    nt = fx[n]
    for i in (n-1):-1:1
        nt = fx[i] + (t - x[i]) * nt
    end
    return nt
end
```

Złożoność czasowa algorytmu

Złożoność czasowa algorytmu wynosi $O(n)$, ponieważ jedyna pętla wykonuje dokładnie n iteracji, w każdej z nich wykonując stałą liczbę operacji (dodawanie i mnożenie). Ponieważ operacje te są wykonywane tylko raz, algorytm jest liniowy względem liczby węzłów, co czyni go bardzo efektywnym czasowo.

4 Obliczanie współczynników postaci naturalnej wielomianu Newtona

Założmy, że mamy wielomian interpolacyjny w postaci Newtona, który jest zapisany w następującej formie:

$$p(x) = \sum_{k=0}^n c_k q_k(x) = \sum_{k=0}^n f[x_0, \dots, x_k] \prod_{j=0}^{k-1} (x - x_j),$$

gdzie $f[x_0, \dots, x_k]$ to ilorazy różnicowe, a $q_k(x)$ to odpowiednie wielomiany bazowe.

Zauważmy, że współczynniki c_k są ilorazami różnicowymi, a współczynniki postaci naturalnej są związane z tymi ilorazami. Na przykład:

- Współczynnik a_n w postaci naturalnej odpowiada $c_n = f[x_0, \dots, x_n]$,
- Współczynnik a_{n-1} przy x^{n-1} to $a_{n-1} = c_{n-1} - x_n a_n$,
- Kolejne współczynniki można obliczać iteracyjnie, stosując wzory rekurencyjne.

Postać Newtona wielomianu i przekształcenie do postaci naturalnej

Przekształćmy postać Newtona wielomianu do postaci naturalnej, posługując się wielomianami $w_i(x)$, które są definiowane następująco:

$$\begin{aligned} w_n(x) &= f[x_0, \dots, x_n] = c_n, \\ w_{n-1}(x) &= c_{n-1} - x_n w_n(x), \\ w_{n-2}(x) &= c_{n-2} - x_{n-2} w_{n-1}(x) + x w_n(x), \\ &\vdots \end{aligned}$$

$$w_0(x) = f[x_0] + (x - x_0)(f[x_0, x_1] + (x - x_1)(f[x_0, x_1, x_2] + \dots)).$$

Zauważmy, że współczynniki przy x^n w postaci Newtona odpowiadają współczynnikom c_n , podczas gdy współczynniki przy niższych potęgach x są obliczane iteracyjnie, z wykorzystaniem wzorów na współczynniki a_i .

Wzory rekurencyjne na współczynniki

Aby obliczyć współczynniki a_i postaci naturalnej, stosujemy następujące wzory rekurencyjne:

$$\begin{aligned} a_n &= c_n, \\ a_{n-1} &= c_{n-1} - x_n \cdot a_n, \end{aligned}$$

$$a_{n-2} = c_{n-2} - x_{n-2} \cdot a_{n-1} + x \cdot a_n,$$

$\vdots$

Zauważmy, że współczynniki przy wyższych potęgach są obliczane przez aktualizowanie poprzednich wartości współczynników. Na przykład, współczynniki a_{n-1} i a_{n-2} są aktualizowane w kolejnych iteracjach.

Wyjaśnienie algorytmu na podstawie wielomianów $w_i(x)$

Aby lepiej zrozumieć algorytm obliczania współczynników postaci naturalnej, warto rozpatrzyć sposób, w jaki kolejne współczynniki a_i są obliczane iteracyjnie, korzystając z wielomianów $w_i(x)$.

Przypomnijmy definicje:

- Wielomian $w_n(x)$ odpowiada najwyższemu stopniowi wielomianu i ma postać $w_n(x) = f[x_0, \dots, x_n] = c_n$,
- Wielomian $w_k(x)$ dla $k < n$ jest zdefiniowany jako:

$$w_k(x) = c_k + (x - x_k)w_{k+1}(x),$$

gdzie $c_k = f[x_0, \dots, x_k]$ to iloraz różnicowy.

Wielomian $N_n(x)$ w postaci Newtona jest zatem równy $w_0(x)$. Teraz przeanalizujemy, jak kolejne współczynniki są obliczane.

Początkowe obliczenia dla $w_n(x)$ oraz $w_{n-1}(x)$

1. $w_n(x)$:

$$w_n(x) = c_n = a_n^{(0)},$$

więc początkowa wartość współczynnika $a_n^{(0)}$ jest równa c_n , a wszystkie pozostałe współczynniki są takie, jak na początku (czyli równe c_k).

2. $w_{n-1}(x)$:

$$w_{n-1}(x) = c_{n-1} + (x - x_{n-1})w_n = c_{n-1} - x_{n-1} \cdot w_n + x \cdot w_n.$$

Podstawiając $w_n = a_n^{(0)} = c_n$, otrzymujemy:

$$w_{n-1}(x) = c_{n-1} - x_{n-1} \cdot a_n^{(0)} + x \cdot a_n^{(0)}.$$

Wartości współczynników:

- $a_n^{(1)} = a_n^{(0)} = c_n$,
- $a_{n-1}^{(1)} = c_{n-1} - x_{n-1} \cdot a_n^{(0)}$, czyli:

$$a_{n-1}^{(1)} = a_{n-1}^{(0)} - x_{n-1} \cdot a_n^{(0)}.$$

Pozostałe współczynniki $a_k^{(1)}$ pozostają niezmiennicze.

Obliczenia dla $w_{n-2}(x)$

1. $w_{n-2}(x)$:

$$w_{n-2}(x) = c_{n-2} + (x - x_{n-2})w_{n-1}(x).$$

Podstawiając $w_{n-1}(x)$ z poprzedniego kroku:

$$w_{n-2}(x) = c_{n-2} + (x - x_{n-2}) \left(c_{n-1} - x_{n-1} \cdot a_n^{(0)} + x \cdot a_n^{(0)} \right).$$

Rozwijamy ten wzór:

$$w_{n-2}(x) = c_{n-2} - x_{n-2} \cdot \left(c_{n-1} - x_{n-1} \cdot a_n^{(0)} + x \cdot a_n^{(0)} \right) + x \cdot \left(c_{n-1} - x_{n-1} \cdot a_n^{(0)} + x \cdot a_n^{(0)} \right).$$

Ostatecznie:

$$w_{n-2}(x) = c_{n-2} - x_{n-2} \cdot \left(c_{n-1} - x_{n-1} \cdot a_n^{(0)} \right) + x \cdot \left(c_{n-1} - x_{n-1} \cdot a_n^{(0)} \right) + x^2 \cdot a_n^{(0)}.$$

Wartości współczynników:

- $a_n^{(2)} = a_n^{(1)} = c_n,$
- $a_{n-1}^{(2)} = a_{n-1}^{(1)} - x_{n-2} \cdot a_n^{(1)},$
- $a_{n-2}^{(2)} = c_{n-2} - x_{n-2} \cdot a_{n-1}^{(1)}.$

Wzory dla współczynników

Na podstawie powyższych obliczeń, dla dowolnego i -tego etapu obliczeń możemy zapisać wzory dla współczynników $a_{n-k}^{(i)}$ w następujący sposób:

- $a_n^{(i)} = c_n,$
- $a_{n-1}^{(i)} = a_{n-1}^{(i-1)} - x_{n-i} \cdot a_n^{(i-1)},$
- $a_{n-2}^{(i)} = a_{n-2}^{(i-1)} - x_{n-i} \cdot a_{n-1}^{(i-1)},$
- Współczynniki dla wyższych potęg $a_k^{(i)}$ są obliczane podobnie.

Podsumowanie

Na podstawie tych obliczeń widać, że każda iteracja aktualizuje współczynniki przy coraz niższych potęgach x , w oparciu o wzory rekurencyjne, które uwzględniają wcześniej obliczone wartości współczynników. Dla każdej iteracji i , współczynniki $a_{n-k}^{(i)}$ są obliczane na podstawie wartości c_k i wcześniej zaktualizowanych współczynników.

Implementacja algorytmu

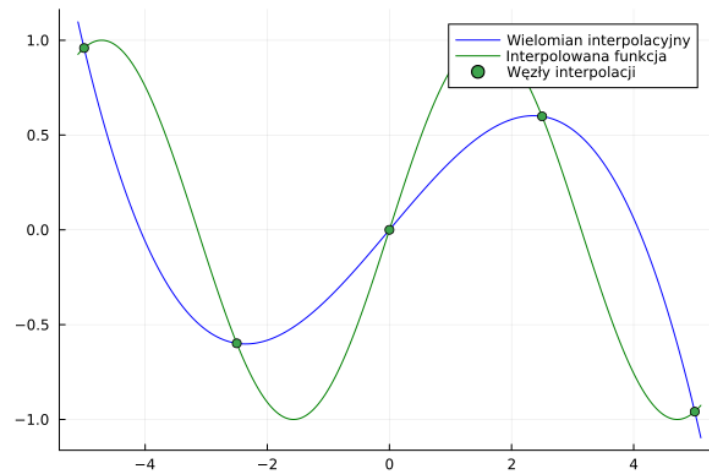
Algorytm obliczania współczynników postaci naturalnej na podstawie ilorazów różnicowych w postaci Newtona można zaimplementować w następujący sposób:

```
function naturalna(x::Vector{Float64}, fx::Vector{Float64})
    n = length(x)
    a = copy(fx)

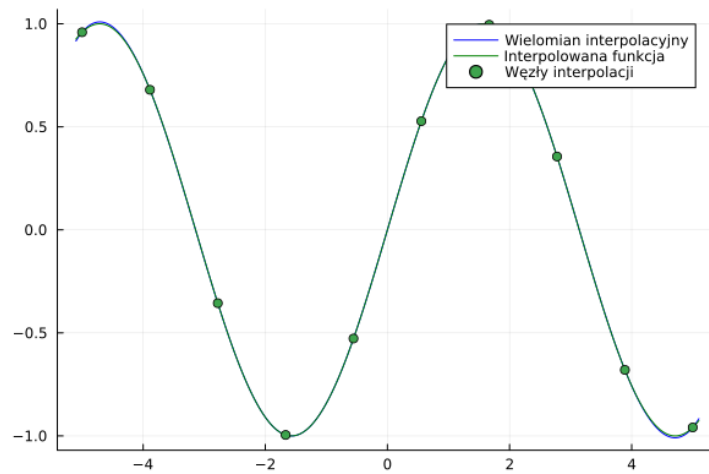
    for i in n-1:-1:1
        for j in i:(n-1)
            a[j] = a[j] - x[i] * a[j+1]
        end
    end

    return a
end
```

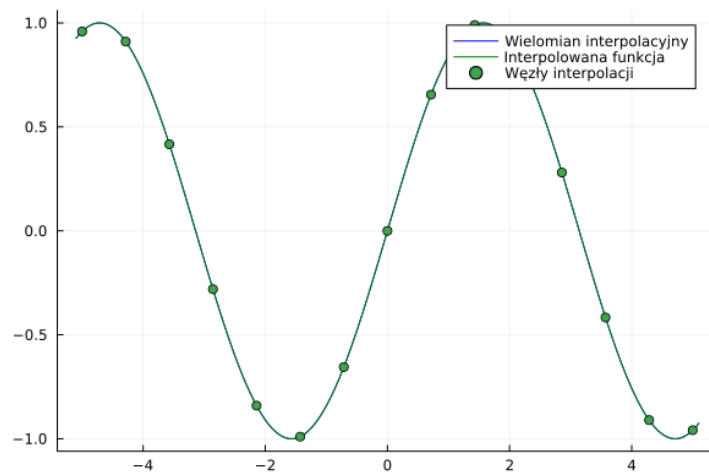
Wyniki algorytmu dla funkcji $f(x) = \sin(x)$ przedstawiono na poniższych wykresach.



Rysunek 2: Wyznaczanie współczynników wielomianu interpolowanego funkcji $\sin(x)$ na przedziale $[-5, 5]$ z $n = 5$



Rysunek 3: Wyznaczanie współczynników wielomianu interpolowanego funkcji $\sin(x)$ na przedziale $[-5, 5]$ z $n = 10$



Rysunek 4: Wyznaczanie współczynników wielomianu interpolowanego funkcji $\sin(x)$ na przedziale $[-5, 5]$ z $n = 15$

Dla $n = 5$ wyniki nie są jeszcze dokładne. Zwiększenie do $n = 10$ powoduje znaczną poprawę dokładności wyniku, co pozostaje niezmienione przy $n = 15$.

Złożoność obliczeniowa

Złożoność czasowa algorytmu wynosi $O(n^2)$, ponieważ mamy dwie zagnieżdżone pętle, z których jedna iteruje przez wszystkie elementy, a druga przechodzi przez

elementy w danym przedziale. Każda operacja w pętli jest operacją stałą, więc całkowita złożoność jest kwadratowa względem liczby węzłów n .

5 Testowanie algorytmu interpolacyjnego

Algorytm interpolacji funkcji przy użyciu wielomianu Newtona został przetestowany na czterech różnych funkcjach w różnych przedziałach. Testy obejmowały także różne stopnie wielomianu interpolacyjnego, co miało na celu sprawdzenie wpływu zwiększania stopnia wielomianu na dokładność interpolacji. Zastosowane funkcje testowe to:

- **Funkcja wykładnicza:** $f_1(x) = \exp(x)$, na przedziale $[0, 1]$.
- **Funkcja trygonometryczno-podobna:** $f_2(x) = x^2 \sin(x)$, na przedziale $[-1, 1]$.
- **Funkcja modułowa:** $f_3(x) = |x|$, na przedziale $[-1, 1]$, gdzie może wystąpić zjawisko rozbieżności.
- **Funkcja racjonalna:** $f_4(x) = \frac{1}{1+x^2}$, na przedziale $[-5, 5]$, gdzie występuje zjawisko Runge'go.

Wszystkie funkcje zostały przetestowane dla wielomianów stopni $n = 5, 10, 15$.

Twierdzenie o błędzie interpolacji funkcji

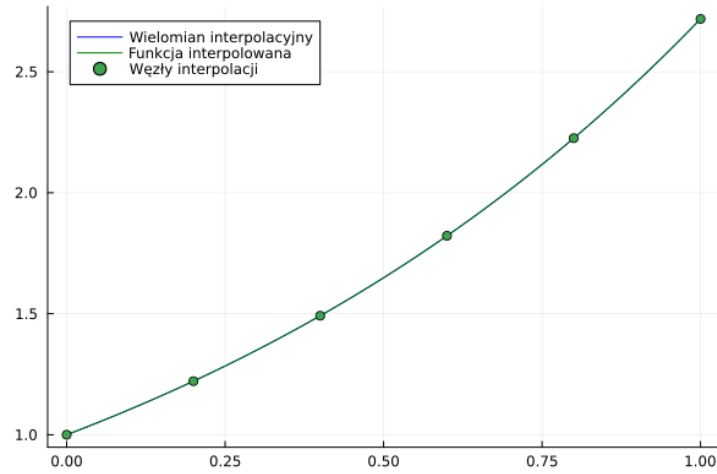
Błąd interpolacji w punkcie x dla funkcji $f(x)$ jest dany przez wzór:

$$r(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \cdot p_n(x)$$

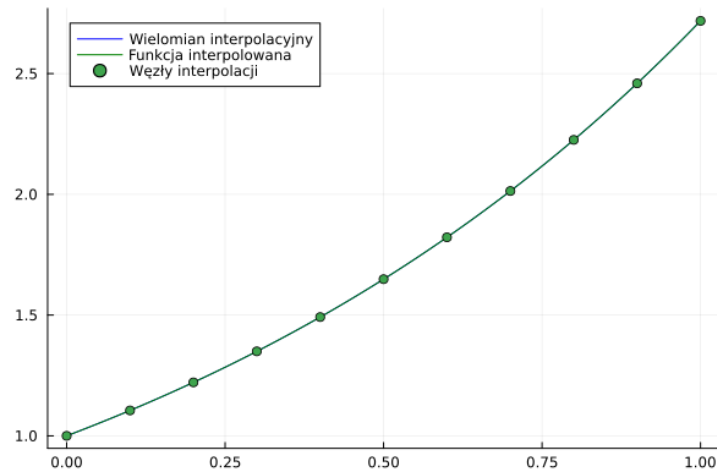
gdzie:

- ξ to liczba z przedziału (a, b) ,
- $p_n(x)$ to iloczyn $(x - x_0)(x - x_1) \cdots (x - x_n)$,
- $f^{(n+1)}(\xi)$ to pochodna $n + 1$ -ego rzędu w punkcie ξ .

Test 1: $f(x) = e^x$



Rysunek 5: Interpolacja funkcji e^x na przedziale $[0, 1]$ z $n = 5$

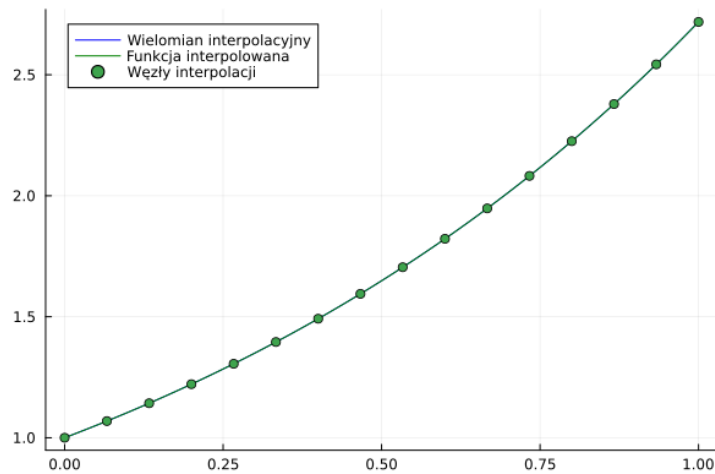


Rysunek 6: Interpolacja funkcji e^x na przedziale $[0, 1]$ z $n = 10$

Przeprowadzona interpolacja funkcji e^x okazała się być bardzo dokładna. Funkcja $f(x) = e^x$ ma pochodne wszystkich rzędów równe e^x , dlatego:

$$f^{(n+1)}(x) = e^x$$

Błąd interpolacji w punkcie x można zatem zapisać jako:



Rysunek 7: Interpolacja funkcji e^x na przedziale $[0, 1]$ z $n = 15$

$$|r(x)| \leq \frac{e}{(n+1)!} \cdot |p_n(x)|$$

Iloczyn odległości między węzłami

Dla węzłów równoodległych w przedziale $[0, 1]$, węzły przyjmują postać:

$$x_k = 0 + k \cdot h, \quad h = \frac{1}{n}, \quad k = 0, 1, \dots, n$$

Dla punktu $x' \approx 0.5$, iloczyn $p_n(x')$ jest maksymalny i wynosi:

$$p_n(x') = \prod_{i=0}^n |x' - x_i|$$

Obliczenia błędu dla różnych n

Dla $n = 5$

$$(n+1)! = 6! = 720$$

Zakładając, że $p_5(x') \approx 0.00225$, mamy:

$$|r(x)| \leq \frac{2.718}{720} \cdot 0.00225 \approx 3.8 \times 10^{-3}$$

Dla $n = 10$

$$(n+1)! = 11! = 39916800$$

Zakładając, że $p_{10}(x') \approx 0.0000036$, mamy:

$$|r(x)| \leq \frac{2.718}{39916800} \cdot 0.0000036 \approx 6.8 \times 10^{-8}$$

Dla $n = 15$

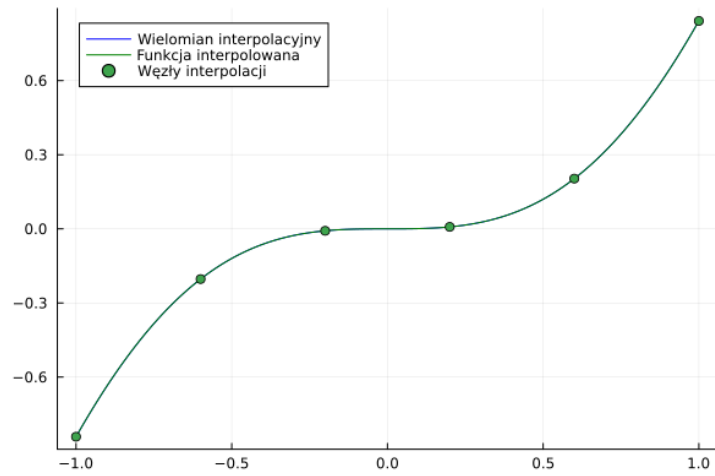
$$(n+1)! = 16! = 20922789888000$$

Zakładając, że $p_{15}(x') \approx 0.0000000000008$, mamy:

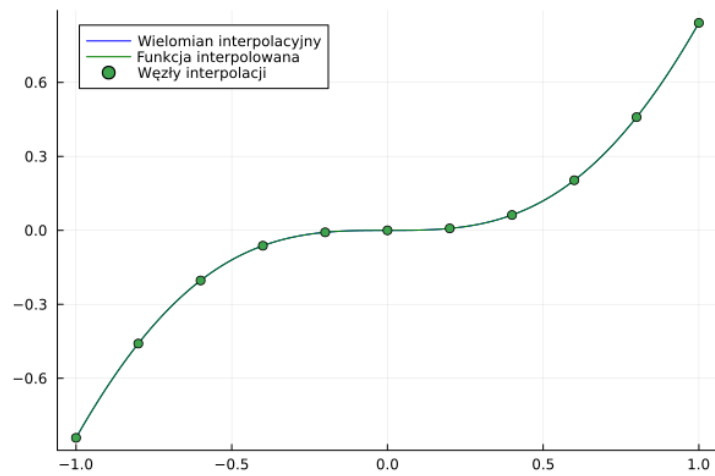
$$|r(x)| \leq \frac{2.718}{20922789888000} \cdot 0.0000000000008 \approx 1.3 \times 10^{-13}$$

Widzimy, że błąd interpolacji maleje wykładniczo, co potwierdza, że wielomian interpolacyjny staje się coraz bardziej precyzyjny przy większych wartościach n .

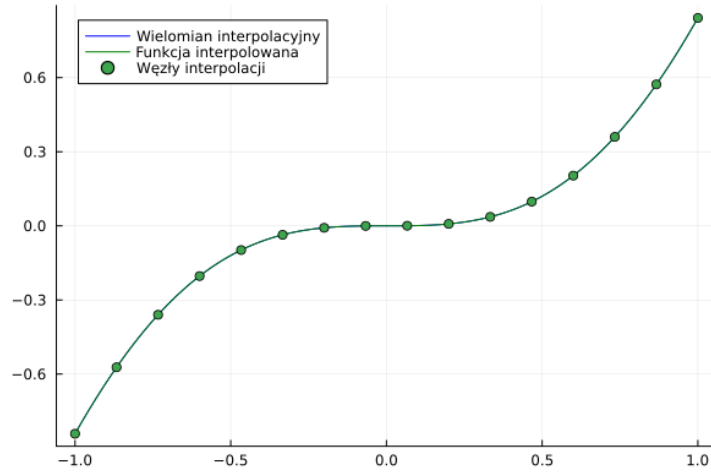
Test 2: $f(x) = x^2 \cdot \sin(x)$



Rysunek 8: Interpolacja funkcji $f(x) = x^2 \cdot \sin(x)$ na przedziale $[-1, 1]$ z $n = 5$



Rysunek 9: Interpolacja funkcji $f(x) = x^2 \cdot \sin(x)$ na przedziale $[-1, 1]$ z $n = 10$



Rysunek 10: Interpolacja funkcji $f(x) = x^2 \cdot \sin(x)$ na przedziale $[-1, 1]$ z $n = 15$

W tym przypadku, interpolacja funkcji również okazała się być bardzo dokładna. b

Zbadajmy błąd interpolacji dla funkcji $f(x) = x^2 \sin(x)$ na przedziale $[-1, 1]$ oraz dla różnych wartości n .

Dla funkcji $f(x) = x^2 \sin(x)$, mamy dane dla wyższych pochodnych na przedziale:

$$f^{(6)}(x) = -x^2 \sin(x) + 12x \cos(x) + 30 \sin(x),$$

$$f^{(11)}(x) = -x^2 \cos(x) - 22x \sin(x) + 110 \cos(x),$$

$$f^{(16)}(x) = x^2 \sin(x) - 32x \cos(x) - 240 \sin(x).$$

Oraz wartości maksymalne tych pochodnych:

$$\max |f^{(6)}(x)| < 32, \quad \max |f^{(11)}(x)| = 110, \quad \max |f^{(16)}(x)| < 243.$$

Obliczenia błędu interpolacji dla $n = 5, 10, 15$

Mając powyższe informacje, możemy obliczyć górne granice błędu interpolacji.

1. Dla $n = 5$

- Mamy:

$$f^{(6)}(x) \text{ z maksymalną wartością } \max |f^{(6)}(x)| < 32,$$

$$(n+1)! = 6! = 720,$$

$$|p_5(x)| \leq 1 \quad \text{dla } x \in [-1, 1].$$

- Zatem błąd interpolacji w przypadku $n = 5$ jest oszacowany przez:

$$|r(x)| \leq \frac{32}{720} \cdot |p_5(x)|$$

Zakładając $|p_5(x)| \leq 1$, mamy:

$$|r(x)| \leq \frac{32}{720} = 4.44 \times 10^{-2}.$$

2. Dla $n = 10$

- Mamy:

$$f^{(11)}(x) \text{ z maksymalną wartością } \max |f^{(11)}(x)| = 110,$$

$$(n+1)! = 11! = 39916800,$$

$$|p_{10}(x)| \leq 1 \text{ dla } x \in [-1, 1].$$

- Zatem błąd interpolacji w przypadku $n = 10$ jest oszacowany przez:

$$|r(x)| \leq \frac{110}{39916800} \cdot |p_{10}(x)|$$

Zakładając $|p_{10}(x)| \leq 1$, mamy:

$$|r(x)| \leq \frac{110}{39916800} = 2.76 \times 10^{-6}.$$

3. Dla $n = 15$

- Mamy:

$$f^{(16)}(x) \text{ z maksymalną wartością } \max |f^{(16)}(x)| < 243,$$

$$(n+1)! = 16! = 20922789888000,$$

$$|p_{15}(x)| \leq 1 \text{ dla } x \in [-1, 1].$$

- Zatem błąd interpolacji w przypadku $n = 15$ jest oszacowany przez:

$$|r(x)| \leq \frac{243}{20922789888000} \cdot |p_{15}(x)|$$

Zakładając $|p_{15}(x)| \leq 1$, mamy:

$$|r(x)| \leq \frac{243}{20922789888000} = 1.16 \times 10^{-11}.$$

Błąd interpolacji dla funkcji $f(x) = x^2 \sin(x)$ na przedziale $[-1, 1]$ zmniejsza się wraz ze wzrostem stopnia wielomianu interpolacyjnego n , a wartości te są bardzo małe:

$$\text{Dla } n = 5, \quad err \approx 4.44 \times 10^{-2},$$

$$\text{Dla } n = 10, \quad err \approx 2.76 \times 10^{-6},$$

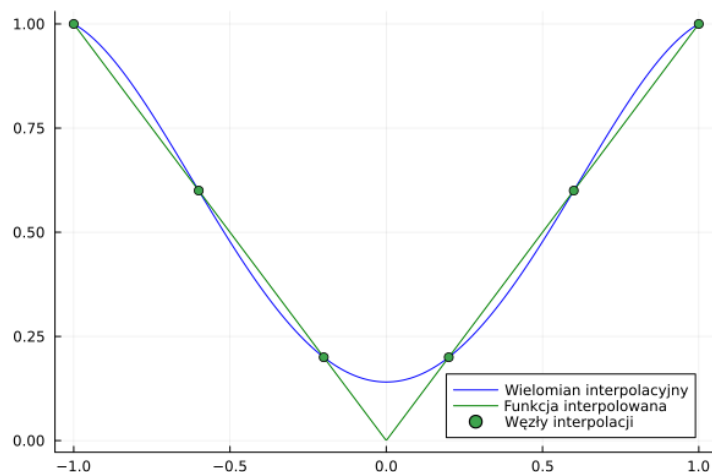
$$\text{Dla } n = 15, \quad err \approx 1.16 \times 10^{-11}.$$

Efekt Runge'go

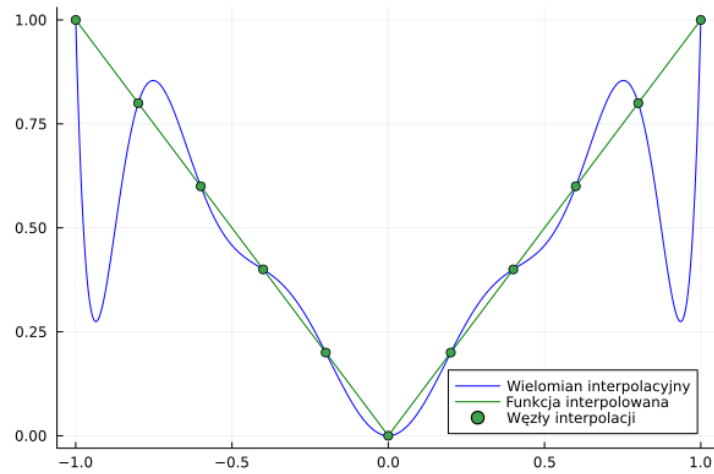
Efekt Runge'go polega na tym, że błąd interpolacji przy użyciu wielomianów rośnie dramatycznie w miarę dodawania kolejnych punktów interpolacyjnych, szczególnie gdy punkty są rozmieszczone równomiernie na dużym przedziale. Zamiast poprawiać przybliżenie funkcji, większa liczba punktów prowadzi do drastycznego wzrostu oscylacji w interpolowanym wielomianie, zwłaszcza na końcach przedziału.

Efekt ten jest szczególnie widoczny, gdy interpolujemy za pomocą wielomianów wysokich stopni przy równoodległych punktach lub gdy interpolowana funkcja jest nieciągła.

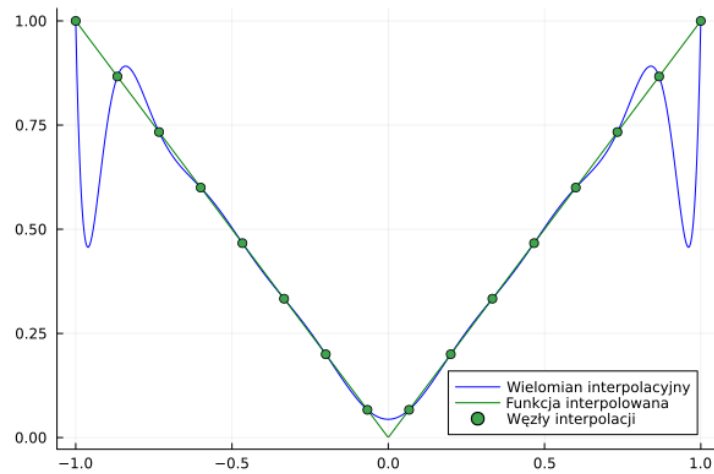
Test 3: $f(x) = |x|$



Rysunek 11: Interpolacja funkcji $|x|$ na przedziale $[-1, 1]$ z $n = 5$



Rysunek 12: Interpolacja funkcji $|x|$ na przedziale $[-1, 1]$ z $n = 10$

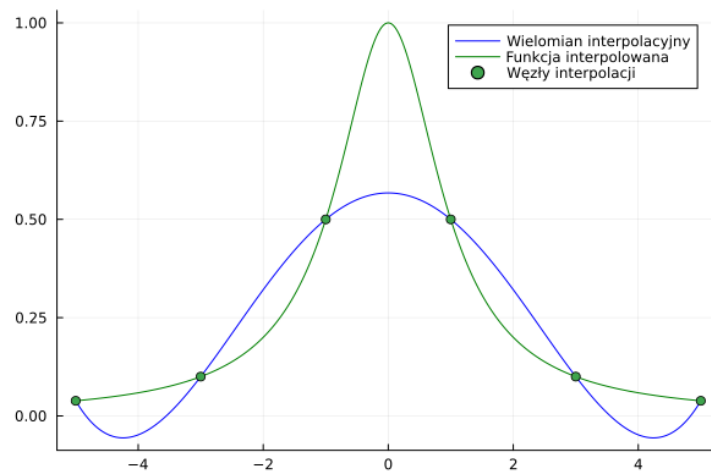


Rysunek 13: Interpolacja funkcji $|x|$ na przedziale $[-1, 1]$ z $n = 15$

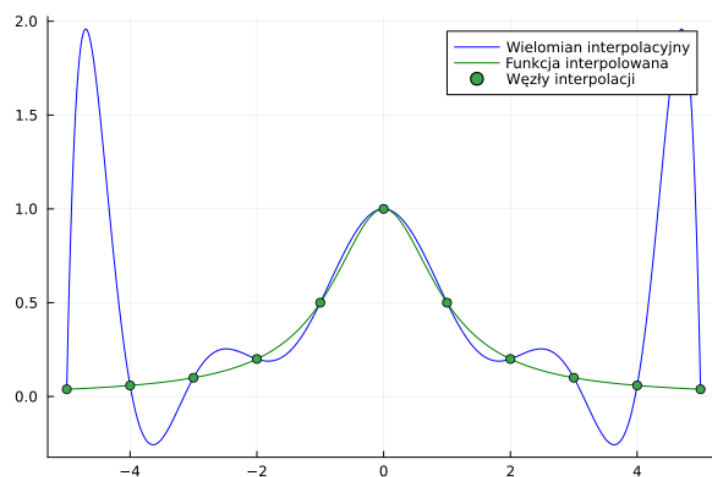
Funkcja $f(x) = |x|$ charakteryzuje się nieciągłością pierwszej pochodnej w punkcie $x = 0$, co powoduje, że jest niediferencjowalna w tym miejscu. Przy interpolacji wielomianowej funkcji $|x|$ pojawia się efekt Runge'go, szczególnie w okolicach $x = 0$. Zjawisko to polega na tym, że wraz ze wzrostem liczby punktów interpolacyjnych (i stopnia wielomianu) błąd interpolacji zamiast maleć, zaczyna rosnąć, zwłaszcza w pobliżu $x = 0$. Wielomian interpolacyjny stara się gładko odwzorować funkcję, ale z powodu nieciągłości pochodnej w tym punkcie wprowadza oscylacje, prowadząc do znacznych odchyśleń od wartości rzeczywistej.

stej funkcji. W rezultacie, większa liczba punktów interpolacyjnych nie poprawia jakości przybliżenia, a wręcz pogarsza je w rejonie punktu nieciągłości. Przy interpolacji funkcji za pomocą wielomianów, zwłaszcza gdy stopień wielomianu jest wysoki, wielomian próbuje jak najlepiej dopasować się do wszystkich punktów interpolacyjnych. W przypadku równomiernie rozmieszczonych punktów na dużym przedziale, wielomian, by sprostać wszystkim wymaganiom interpolacyjnym, zaczyna wykazywać oscylacje w pobliżu końców przedziału. To prowadzi do znacznych błędów na krańcach, mimo że w samym środku przedziału przybliżenie jest stosunkowo dokładne.

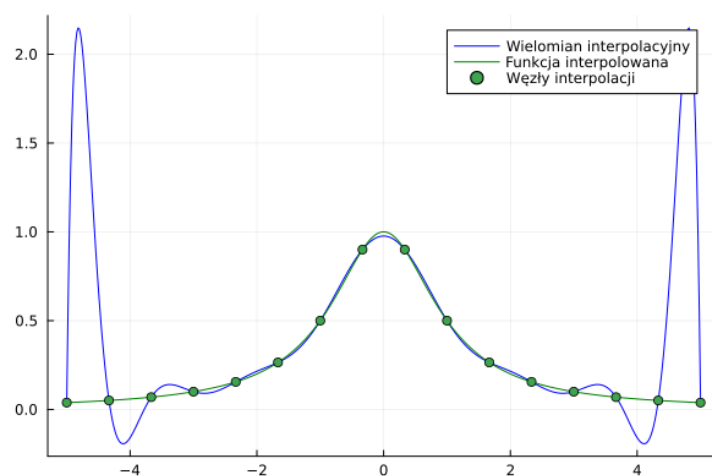
Test 4: $\frac{1}{1+x^2}$



Rysunek 14: Interpolacja funkcji $\frac{1}{1+x^2}$ na przedziale $[-5, 5]$ z $n = 5$



Rysunek 15: Interpolacja funkcji $\frac{1}{1+x^2}$ na przedziale $[-5, 5]$ z $n = 10$



Rysunek 16: Interpolacja funkcji $\frac{1}{1+x^2}$ na przedziale $[-5, 5]$ z $n = 15$

Funkcja $f(x) = \frac{1}{1+x^2}$ jest przykładem funkcji, która, w przeciwieństwie do $|x|$, jest gładka i różniczkowalna na całym zbiorze liczb rzeczywistych. Dodatkowo, funkcja ta jest ograniczona i ma asymptoty w nieskończoności, co oznacza, że wartości funkcji maleją do zera, ale nigdy go nie osiągną. Niemniej jednak interpolacja wielomianowa tej funkcji może również prowadzić do problemów związanych z efektem Runge'go, szczególnie gdy używamy dużej liczby punktów interpolacyjnych.

Mimo swojej gładkości, funkcja $f(x)$ może nadal wykazywać zjawisko efektu

Runge'go przy interpolacji wielomianowej, jeśli punkty interpolacyjne są rozmieszczone równomiernie na dużym przedziale. Wielomian interpolacyjny może próbować odwzorować funkcję w sposób, który prowadzi do oscylacji na krańcach przedziału, co skutkuje dużymi odchyleniami w miejscach, gdzie funkcja jest "płaska" (bliska zeru).

Mamy również, że jeśli liczba równomiernie rozmieszczonych węzłów zmierza do nieskończoności, to maksymalny błąd interpolacji naszej funkcji odpowiednim wielomianem również zmierza do nieskończoności.

Jeśli założymy, że funkcja $f(x)$ jest funkcją gładką, ale jej wyższe pochodne rosną w sposób szybki (np. dla funkcji $f(x) = \frac{1}{1+x^2}$, mamy do czynienia z rosnącymi pochodnymi), to błąd interpolacji będzie rósł. Dla $n \rightarrow \infty$, produkt

$$\prod_{i=0}^n (x - x_i)$$

rośnie w sposób wykładniczy (związane jest to z tym, że węzły są równomiernie rozmieszczone). Jednocześnie, jeśli $f^{(n+1)}(x)$ rośnie, błąd interpolacji nie tylko nie maleje, ale może rosnąć do nieskończoności. Dowód formalny będzie opierał się na analizie tempa wzrostu pochodnych i na zachowaniu iloczynu

$$\prod_{i=0}^n (x - x_i),$$

który dla dużych n będzie prowadził do nieograniczonego wzrostu błędu.

Poprawa interpolacji za pomocą wielomianów Czebyszewa

Jednym ze sposobów poprawy jakości interpolacji w przypadku funkcji, które mogą wykazywać efekt Runge'go, jest zastosowanie wielomianów Czebyszewa. Zamiast stosować równomiernie rozmieszczone punkty interpolacyjne, możemy wykorzystać węzły Czebyszewa, które są rozkładają się w sposób nieliniowy, co pozwala na lepsze odwzorowanie funkcji w miejscach, gdzie funkcja zmienia się szybciej, a także minimalizuje oscylacje na końcach przedziału.